EMPÓRIO DA MÚSICA

Generative AI support assistant

Technical challenge - AI Engineer

Python, structured queries, and policy retrieval with a locally executed model.

Gabriel Silva

Model: Qwen3 8B through Ollama. No paid inference API.

Source revision: 12de53467bc3e5565dd0d7d0bd27b08ee8b4e8e9

Revision timestamp: 2026-11-03T22:10:47-03:00

I prepared this submission from the versioned README, technical documentation, and five labelled translations of recorded conversations.

github.com/gsssilva/emporio-musica-ai

Contents

Empório da Música - AI customer support assistant 3

Architecture and design rationale 8

Structured-data ingestion 10

Queries and commercial rules 12

Policy retrieval 13

Agent, sessions, and privacy 14

Local HTTP API 16

Chat interface 18

Evaluation and limitations 20

Requirements and source ambiguities 22

Catalogue search 24

Promotion and PIX without stacking 25

Return request with missing context 26

Order lookup with simulated identity 27

Out-of-scope question 28

Empório da Música - AI customer support assistant

I built this Python application for Artefact's **AI Engineer - Full-Stack Generative AI Applications** technical challenge. The assistant answers questions about a fictional music store using the supplied product data and policy manual. My focus is a working, inspectable agent: accurate calculations, traceable evidence, clear limits, and a straightforward way to test conversations.

Public repository: gsssilva/emporio-musica-ai.

The runtime uses **Qwen3 8B through local Ollama**, FastAPI, and SQLite. It requires no paid inference API, API key, or subscription. Model downloads require internet access; inference uses the host computer's memory, processing power, and electricity.

Scope

I implemented validated ingestion of all six CSV files, five typed tools, policy retrieval with section and page references, isolated conversation sessions, and a browser chat. Swagger is available for inspecting the HTTP contract. The application does not execute payments, approve returns, update orders, or connect to WhatsApp, an ERP, or a carrier.

The supplied data, policy quotations, customer-facing interface, and agent persona remain in Portuguese because the fictional store operates in Brazil. I wrote the submission documentation in English. The five English conversation examples are explicitly labelled translations; the original model outputs remain in the evaluation JSON.

Run locally

Requirements: Python 3.11+, Git, Ollama, and enough storage for the approximately 5.2 GB model download. The validated environment is Windows with 16 GB RAM and an RTX 4060 with 8 GB VRAM. Other hardware may be slower, particularly without a GPU. Node.js is needed only for the optional frontend tests.

Run these commands in PowerShell:

```
git clone https://github.com/gsssilva/emporio-musica-ai.git
cd emporio-musica-ai
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.lock
.\.venv\Scripts\python.exe -m pip install -e . --no-deps
.\.venv\Scripts\python.exe -m emporio_ai check-data
.\.venv\Scripts\python.exe -m emporio_ai ingest
.\.venv\Scripts\python.exe -m emporio_ai index-policies
```

I keep the original inputs in `data/`. Generated databases, conversation history, model files, and local configuration belong in `runtime/`, which Git ignores. Ingestion replaces the catalogue snapshot transactionally and rejects incompatible databases.

Start the model

Install Ollama, or place its official portable Windows package in `runtime/tools/ollama/`. Stop any other Ollama server using port 11434 before starting this project's server. In one terminal:

```
powershell -ExecutionPolicy Bypass -File scripts/start-model.ps1
```

The script configures its process to use loopback, disable Ollama cloud features, and store models in `runtime/ollama-models`. In another terminal, download the model once:

```
ollama pull qwen3:8b
```

For the portable installation without Ollama on PATH, use `.\runtime\tools\ollama\ollama.exe pull qwen3:8b`. No inference-provider login or key is required. After the download, model requests stay local.

Start the API and chat

```
.\.venv\Scripts\python.exe -m uvicorn emporio_ai.api:app --host 127.0.0.1 --port 8000 --no-access-log
```

Open the chat. It creates a conversation automatically. Enter sends a message; Shift+Enter adds a line. Sources appear below each answer, and the technical panel shows the actual tool trace. **Nova conversa** deletes the current conversation after confirmation and starts another one.

The tab stores its token and displayed messages in `sessionStorage` so a reload can resume the conversation. The server keeps history for up to 24 hours. The first inference may take longer while the model loads. `GET /chat/health` checks model availability without generating an answer. Stop the API and model terminals with Ctrl+C.

`.env` is optional; the defaults require no credentials. See `.env.example` for configuration. Order access is blocked by default. For the fictional order demonstration only, set `EMPORIO_DEMO_CUSTOMER_ID=2` on the server and restart the API. This simulated identity is not production authentication and cannot be chosen by the browser or model.

To inspect the API in Swagger:

1. Execute `POST /sessions` with an empty body or `{}`.
2. Copy the returned `session_id` into **Authorize** as the Bearer token.
3. Execute `POST /chat` with the following body.
4. Inspect `answer`, `sources`, `tool_trace`, `generation_mode`, and `status`.
5. Execute `DELETE /sessions/current` to remove the conversation.

```
{"message":"Quais violões estão disponíveis até R$ 1000?"}
```

The session token is generated by this application; it is not a paid API key or proof of customer identity.

Technical decisions

Agent approach and framework

I chose a hybrid approach: function calling for structured data and commercial calculations, and retrieval-augmented generation for store policies. Prices and stock require exact queries; return procedures need the surrounding policy text. I keep authorization and monetary arithmetic in Python rather than delegating them to the model.

I implemented the tool loop directly in Python with Pydantic validation and explicit limits. For this small application, I preferred an inspectable execution path over adding an orchestration framework. FastAPI provides request validation, HTTP endpoints, and OpenAPI documentation. The closed tool registry exposes `search_products`, `get_product`, `quote_payment`, `search_policies`, and `get_order_status`. It exposes no arbitrary SQL or commercial write operation.

Model and provider

I selected Qwen3 8B through Ollama to support tool calls without a paid inference service on the available hardware. I restricted the provider to loopback and disabled cloud features. The trade-off is a substantial download and hardware-dependent latency. Temperature zero and a fixed seed reduce variation but do not guarantee identical results across machines or runtime versions.

Retrieval and prompt

I chose SQLite FTS5/BM25 for an eight-page manual. Section-based extraction produces 29 chunks, preserving page references and the original document hash. Title weighting, accent normalization, controlled term expansion, and related-section retrieval provide a small, traceable baseline. I would compare embeddings against this baseline before introducing another retrieval component.

I defined a Portuguese system prompt for the store persona, scope, evidence requirements, and missing-information handling. Prices and policy text come from tools rather than being copied into the prompt. User messages and retrieved content do not receive system authority. Responses identify the prompt version. Server-generated sources establish which records were consulted; they do not prove that every sentence interprets them correctly.

Interface and history

I chose a small HTML/CSS/JavaScript chat served by FastAPI so an evaluator can send messages without managing session tokens manually. I kept the chat prominent and retained Swagger for technical inspection. The additional catalogue view at `/catalogo` is secondary; I did not expand it into an e-commerce application. Assets and fonts are local, and the frontend shares the API's origin.

I persisted history in a separate SQLite database to support follow-up questions without coupling conversations to catalogue imports. Session tokens are random, only their hashes are stored, and sessions expire after 24 hours. The model receives at most three recent user/assistant pairs. This bounds context size but can lose earlier details. A conversation token grants access to that conversation, not to a customer's orders.

Data processing

I preserved the original input bytes and SHA-256 hashes. Before a transactional import, I validate schemas, types, IDs, foreign keys, status values, monetary precision, dates, and promotion conflicts. I use integer cents and basis points with explicit rounding. Historical order totals are preserved instead of being recalculated from current prices.

I extract policies by section without rewriting their content. Ambiguities remain visible: conflicting contacts, shipping at exactly R$ 500, small installments, and missing purchase-channel or actual-receipt dates lead to clarification or human confirmation.

Verification and evidence

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -q
node --test tests/test_chat_ui.cjs
.\.venv\Scripts\python.exe scripts/audit_data.py
.\.venv\Scripts\python.exe scripts/evaluate_agent.py --output runtime/evaluation.json
```

I separate deterministic tests, real-model evaluation, and answer review. The Python suite contains 93 tests covering ingestion, queries, money, retrieval, authorization, sessions, HTTP behavior, assets, and agent safeguards. Ten JavaScript tests exercise client behavior with explicit DOM and HTTP test doubles; they require no npm packages. Neither set measures language-model answer quality.

The recorded real-model evaluation contains 13 scenarios, with 13 passing their defined automatic checks, at revision `986e4e63339539b80168e2d969af94bfc48eee27`. It records model digest, prompt, settings, timings, answers, sources, and trace. That historical run predates the latest regression fixes and interface changes; I do not present it as a fresh evaluation of those changes or as universal accuracy. See evaluation and limitations.

I included five varied conversations, including promotion/payment interaction and a return request with missing context:

- Catalogue search
- Promotion and PIX
- Return eligibility
- Order with simulated identity
- Out-of-scope question

To export a new real run without overwriting the English translations:

```
.\.venv\Scripts\python.exe scripts/evaluate_agent.py --output runtime/evaluation.json --examples
runtime/examples-original
```

Documentation

```
Browser / HTTP -> session -> agent -> local Ollama
                            |-> validated tools -> catalogue SQLite
                            |-> integer monetary calculations
                            |-> FTS5 retrieval -> policy sections and pages
                     <- actual sources and explicit business safeguards
```

- Architecture
- Ingestion and provenance
- Queries and commercial rules
- Policy retrieval
- Agent, sessions, and privacy
- HTTP API
- Chat interface
- Evaluation
- Requirements and source ambiguities
- Delivery preparation
- Machine-readable data audit

`src/emporio_ai/` contains the Python package; `tests/`, deterministic tests; `scripts/`, setup, audit, evaluation, and PDF tooling; `evals/`, scenarios and recorded results; `examples/`, English translations of five recorded conversations. I retained the supplied challenge in `docs/referencias/` for reference.

Known limitations

I treat the CSVs as a local snapshot, not live inventory or tracking. An estimated delivery date is not proof of receipt. The agent cannot approve refunds, open support tickets, or verify carrier updates. Products must be active and in stock to be offered as available.

Lexical retrieval may miss unfamiliar wording. A model can omit conditions or make unsupported suggestions even when its source IDs are valid. Explicit safeguards address known rules and observed failures, not every possible hallucination. The recorded out-of-scope and order examples retain imperfections, discussed alongside their transcripts.

This is a local prototype without production customer authentication, TLS termination, deployment operations, or a concurrency SLA. Only one inference runs at a time. History is bounded; expired or deleted sessions cannot be resumed. I bind the API to loopback rather than exposing it publicly.

What I would do with more time

1. Expand the evaluation set with unseen paraphrases, multi-turn conversations, conflicting instructions, and policy-retrieval checks. I would review answer correctness separately from tool and keyword checks.
2. Compare hybrid lexical/embedding retrieval against the same labelled questions, measuring relevant-section recall before choosing more infrastructure.
3. Add real customer authentication and authorization tests before enabling order access outside the simulated demonstration.
4. Obtain business clarification for ambiguous policies, then add regression tests for the confirmed rules instead of guessing.
5. Measure cold-start and concurrent-request behavior, improve observability without logging conversation secrets, and add browser accessibility checks before any hosted release.

How I used AI assistance

I used Codex throughout the project for source analysis, implementation drafts, debugging, test generation, evaluation support, and documentation editing. I also supplied an AI-assisted HTML layout prototype as a visual reference. AI assistance contributed substantially to the code and documentation; I describe that contribution explicitly rather than presenting the repository as unaided work.

I set the scope and the requirement to avoid paid inference, requested changes to the chat experience, and directed the final review against the challenge. My workflow was iterative: discuss a choice, inspect the proposed change, run the relevant checks, and revisit observed failures. The repository includes the resulting corrections, including promotion/PIX consistency, missing return context, retrieval wording, and conversation reset behavior. Deterministic test results and real-model transcripts provide evidence for the implementation; they do not replace my responsibility to understand and explain it.

Delivery PDF and Git history

Markdown is the source of truth for the delivery document:

```
.\.venv\Scripts\python.exe -m pip install -e ".[docs]"
.\.venv\Scripts\python.exe scripts/build_docs.py
```

Git must be on PATH and the Markdown sources must be committed. The builder records their revision and hashes in `runtime/pdf-sources.json`. It combines this README, technical documentation, and the five examples into the English submission PDF. The PDF artifact is committed after its source revision.

I kept progressive commits and did not use force-push. I follow the recruitment email's submission instructions: an English PDF through the supplied Google Form, with the public repository linked in the document. I will keep the submitted repository unchanged after the applicable delivery cutoff, as required by the challenge. Generating or committing the PDF does not itself submit the application.

Architecture and design rationale

I separated natural-language interpretation from operations that require exact results. The model chooses tools and drafts answers; Python controls access, validates inputs, calculates money, and applies explicit business safeguards.

Request flow

```
Chat / Swagger / HTTP client
  -> FastAPI validation and conversation token
  -> bounded history and server-controlled customer context
  -> preventive policy retrieval when relevant
  -> local Ollama tool selection
  -> closed Pydantic tool registry
     -> read-only catalogue and authorized order queries
     -> integer monetary calculations
     -> FTS5 policy retrieval with section/page provenance
  -> final natural-language answer
  -> evidence checks and identified business safeguards
  -> answer, sources, trace, model and prompt metadata
```

I chose FastAPI for validation and OpenAPI, with a small same-origin browser chat. The agent service can also run without HTTP, as it does in the evaluation script. I did not need a separate orchestration framework for five tools and a bounded loop.

Storage and tools

I use three SQLite databases: catalogue, policy index, and conversations. This keeps catalogue imports from deleting history or policies. The received CSVs contain 191 operational records, so a transactional snapshot import and small local queries fit the problem.

I expose `search_products`, `get_product`, `quote_payment`, `search_policies`, and `get_order_status`. Pydantic rejects extra arguments and invalid types. The server resolves categories from the catalogue and supplies customer identity independently of model arguments. Query connections are read-only, SQL is parameterized, and returned fields are explicit. No tool accepts arbitrary SQL, lists customers, or changes orders.

I parse monetary inputs with `Decimal`, then calculate in integer cents and basis points. This avoids floating-point discrepancies and keeps installment totals exact. I preserve historical order totals rather than applying today's catalogue prices retroactively.

Retrieval and inference

I chose FTS5/BM25 because the manual has eight pages. The index has 29 section-based chunks with original page numbers and a document hash. Controlled term expansion and related sections improve context without rewriting policies. Lexical retrieval remains vulnerable to unfamiliar phrasing; I would benchmark embeddings before adding them.

I run Qwen3 8B through Ollama on loopback, with cloud features disabled and no paid fallback. The provider ignores proxy configuration and rejects remote endpoints. A fixed seed and temperature zero support repeatability, not cross-hardware determinism.

Evidence and safeguards

I keep prompt instructions separate from user and retrieved content. The server attaches actual sources and rejects unknown references. Preventive retrieval and deterministic business rules have explicit trace markers, so they are distinguishable from model-selected tool calls.

Safeguards cover known issues such as status/stock availability, PIX and promotion consistency, promotion disclosure, missing return context, warranty timing, historical delivery estimates, and ambiguous shipping thresholds. They do not prove every sentence is correct. I therefore separate domain tests from real-model evaluation and discuss imperfect outputs openly.

Sessions and operational boundaries

I store only hashes of random conversation tokens. Sessions expire after 24 hours and can be deleted. A transactional lease prevents simultaneous writes within one conversation; an inference semaphore protects the local model. Logs contain operational metadata rather than full messages or session tokens.

A conversation token does not authenticate a store customer. Order access is blocked unless the server supplies a trusted context. The optional fixed demo identity is explicitly simulated. Production authentication, deployment, external commerce integrations, and scalable inference are outside this submission.

Structured-data ingestion

I preserve the original six CSV files and import a validated snapshot with `python -m emporio_ai ingest`. Parsing and SHA-256 hashing use the same bytes, so the provenance refers to exactly what was imported. The original files are not edited.

Validation before writing

I validate column schemas, CSV structure, UTF-8 encoding, required values, positive IDs, SQLite integer limits, duplicate keys, finite non-negative money, cent precision, item quantities, stock, status domains, dates, JSON specifications, and foreign keys. Every order must have an item. More than one active promotion for a product stops the import for clarification.

I reject duplicate `(order_id, product_id)` item keys because the source has no separate line ID or historical unit-price field that would make their meaning unambiguous.

Source field	Stored field	Representation
`price_brl`	`price_cents`	Integer cents
`total_brl`	`total_cents`	Preserved historical total in cents
`discount_percent`	`discount_bps`	Integer basis points; 10.5% becomes 1050
`specs`	`specs_json`	Normalized JSON object with its content preserved
Empty optional order fields	Same field names	`NULL`

I do not turn an estimated delivery date into a receipt date, correct tracking strings, or recalculate order totals from current prices.

Transaction and repeatability

After validation, I enable foreign keys and start `BEGIN IMMEDIATE`. The loader checks the database version, prepares the schema, replaces the snapshot in relationship order, inserts parameterized values, and updates metadata. Only then does it commit. A write failure rolls back both data and metadata. Invalid input is rejected before opening the database.

I reject unknown tables and incompatible schema versions rather than overwriting an unrelated database or performing an implicit migration. The current catalogue schema version is 1. Repeating ingestion replaces records instead of accumulating copies. Policies and sessions use separate databases.

Provenance and checks

`import_metadata` records the schema version, UTC import time, source names, hashes, and row counts. It does not contain conversation text. The operational database itself contains customer records and stays in ignored `runtime/` storage.

The received snapshot contains 9 categories, 50 customers, 65 products, 20 orders, 22 order items, and 25 promotions: 191 records. I use `scripts/audit_data.py` as an independent source check, not a replacement for ingestion validation. Differences between historical totals and current catalogue sums are warnings rather than automatic corrections.

Tests cover invalid inputs, induced write failure and rollback, incompatible databases, and repeat imports. I mark CSVs as `-text` in `.gitattributes` to prevent line-ending normalization from changing their original hashes on another operating system.

Limits

I load the small snapshot in memory. I did not implement incremental ERP ingestion, change-data capture, or an archive of prior snapshots. Those would require a different data lifecycle and are unnecessary for the supplied volume.

Queries and commercial rules

I expose store facts through `StoreQueries`, using SQLite read-only connections and `PRAGMA query_only`. SQL values are parameterized, and each result projects only fields needed by the assistant.

Catalogue

Search supports text, category, budget, availability, and a limit from 1 to 10 results. I normalize accents and selected Portuguese category plurals. Results are ordered by current price and ID, with total-match and truncation indicators.

I define availability as `status == active` and positive stock. Product detail retains the source status so the assistant can explain out-of-stock, discontinued, and upcoming items. A quote for a new purchase requires availability.

Only active promotions affect current prices. Results include list price, discount in basis points, and current price. The search budget applies to that current price without assuming PIX payment. For 65 products, applying the final text and price filters in Python avoids SQLite floating-point arithmetic without adding search infrastructure. A larger catalogue would need indexed search and pagination.

Payment calculations

I calculate in integer cents and basis points, rounding half up. Installment remainders go to the first installments so their sum remains exact.

- PIX: 5% off list price when there is no active promotion; no additional PIX discount on an active promotion.
- Debit and bank slip: one payment at the current price.
- Credit: 1 to 12 installments; installments 4-6 require at least R$ 80 each, and 7-12 require at least R$ 100 each.
- Two or three installments with any installment below R$ 50 return `needs_confirmation`, because section 3.1 is ambiguous at this boundary.
- Combined payment methods are not calculated.

I use manual sections 3, 3.1, and 6.2 as business inputs. A quote is not credit approval or a transaction. Promotional communication must show the original price, discount percentage, and promotional price. The response safeguard and product cards make these values explicit when applicable.

Orders and access

`get_order(order_id, authorized_customer_id=...)` requires customer context supplied by the server. The SQL filters both order and customer. A missing order and another customer's order return the same diagnostic.

I return recorded status, historical total, payment method, tracking, estimated delivery, and items. I omit customer name, email, phone, customer ID, and internal notes. Actual receipt and purchase channel remain unknown. A past estimate does not authorize an invented update.

This filter is authorization logic, not customer authentication. The optional fixed environment identity is a clearly labelled simulation; neither model arguments nor the browser can choose it.

Freshness and verification

Results identify `local_snapshot` and import time. Repeating a query reads the imported database, not an external ERP or carrier.

Tests cover stock and status together, active promotions, non-stacking PIX, installment boundaries and rounding, budgets, truncation, SQL-like text input, order filtering, minimized fields, and blocked writes.

Policy retrieval

I index the supplied manual with `python -m emporio_ai index-policies`. `pypdf` extracts text into a separate policy database so catalogue imports cannot remove the index.

Preparing the source

I segment by document section, preserve lists, and retain all source pages when a section crosses a page break. I remove repeated headers and footers without rewriting policy content. The eight-page manual produces 29 chunks: its cover and 28 sections. I retain the cover because its contact details differ from the institutional table.

Each source includes a stable ID, section, title, pages, document name, and SHA-256 hash of the original bytes. Indexing builds a temporary database and replaces the existing index only after the new version is complete.

Search strategy

I use SQLite FTS5/BM25 with stronger title weighting, accent normalization, stop-word filtering, and controlled Portuguese term expansion. Search terms are sanitized before forming an FTS expression. Search returns at most five primary matches plus related sections, with a total cap of ten.

I explicitly relate return eligibility to exceptions, defects to warranty, payment to promotion rules, and contact sections to the cover and WhatsApp channel. Complaint wording also retrieves the complaint procedure and its related context. These relationships supply evidence; they do not approve a customer's claim.

An unmatched query returns an empty list. A lexical ranking score is not a probability of correctness. I would evaluate relevant-section recall on unseen paraphrases before adopting embeddings or a hybrid retriever.

Verification and limits

I test provenance, section boundaries, cross-page continuity, numbered procedures, related context, invalid queries, rebuilds without duplicates, and complaint paraphrases. Indexing does not call an LLM. The approach remains sensitive to vocabulary, so a retrieved source must still be checked for relevance and correctly interpreted in the final answer.

Agent, sessions, and privacy

I use local Qwen3 8B through Ollama to interpret store questions, choose tools, and draft answers. I keep evidence handling, authorization, arithmetic, and execution limits in the application.

Provider and cost

The provider accepts only HTTP loopback endpoints (`127.0.0.1`, `localhost`, or `::1`), rejects credentials and cloud model tags, and ignores proxy environment variables. The startup script sets `OLLAMA_NO_CLOUD=1`. There is no paid fallback, inference API key, or message-based charge.

The model download is approximately 5.2 GB. I selected it for tool support and the available GPU, accepting hardware-dependent latency and memory requirements. Model availability is not evidence of answer quality.

Execution

1. I accept the user message with bounded session history and server-controlled customer context.
2. Recognized policy questions receive preventive retrieval, identified by `trigger=retrieval`.
3. The model selects from five tools. Pydantic validates types, limits, and extra fields before execution.
4. Tool results and actual sources return to the model as data. Another tool round is allowed within the budget.
5. A final call without tools produces the natural-language answer. The server builds the HTTP envelope and source list.
6. Evidence checks and explicit business safeguards reject or qualify unsupported results.

The registry contains `search_products`, `get_product`, `quote_payment`, `search_policies`, and `get_order_status`. It has no arbitrary SQL, code execution, network browsing, customer listing, or order-writing tool.

I version the system prompt in `src/emporio_ai/prompts/system.txt`. Responses identify its abbreviated hash, provider, and model. The prompt defines the store persona, scope, evidence requirements, and handling of missing information. User and retrieved text cannot grant new tools or customer access.

Search accepts natural category wording and budgets in reais; the server resolves categories and converts money. If the model finds an unavailable item but does not finish searching for alternatives, an explicit server rule can query the same category. Alternatives are real available products, not a claim of technical equivalence.

Business safeguards use `trigger=business_rule`. They address availability, promotional price disclosure, non-stacking PIX, missing receipt/channel context, warranty timing, expired delivery estimates, and shipping at exactly R$ 500. `generation_mode` distinguishes model output, safeguards, authorization blocks, and safe fallback behavior.

Execution limits

I limit each message to 2,000 characters, six tool-selection rounds, eight tool calls, and a final formatting step. The agent has a 240-second total budget, with up to 90 seconds per model request. Context includes at most three recent user/assistant pairs and a global character budget.

The provider uses temperature zero, seed 42, an 8,192-token context, and up to 1,200 output tokens. These settings do not ensure identical results across runtime versions or hardware. Initial model loading can increase latency.

A provider failure returns a controlled error, releases the session, and preserves prior history rather than inserting a fabricated answer. Tool-budget or evidence failures produce a controlled status. I do not retry indefinitely.

Conversation lifecycle

`POST /sessions` creates a random 256-bit token. Only its hash is stored. The caller sends it as `Authorization: Bearer <session_id>` for chat and deletion. Sessions expire after 24 hours, allow at most 20 turns, and have a global capacity of 500. A transactional lease prevents concurrent work in one conversation. A semaphore permits only one local inference at a time.

`DELETE /sessions/current` removes the session and its messages. History is stored in ignored `runtime/sessions.sqlite3`. Operational logs record request ID, model, prompt version, duration, and tool counts; they omit full messages, tool arguments, session tokens, and credentials.

Order access

I distinguish a conversation capability from customer identity. Order requests without trusted customer context are blocked before inference. `EMPORIO_DEMO_CUSTOMER_ID` supplies a fixed fictional identity on the server for demonstration only. The request body and model cannot choose or change it. Production use would require real authentication and authorization.

Grounding limits

A valid source ID establishes that the application consulted that record. It does not prove that every sentence is faithful to it. The model may omit a condition or add an unnecessary suggestion. I use deterministic regression tests for safeguards and a separate real-model evaluation for language behavior; I do not present test-double results as model accuracy.

Local HTTP API

I serve the chat and API on the same local origin. After installing dependencies, importing data, indexing policies, and starting the model, run:

```
.\.venv\Scripts\python.exe -m uvicorn emporio_ai.api:app --host 127.0.0.1 --port 8000 --no-access-log
```

The root `/` serves the chat; `/docs` serves Swagger; `/assets` serves local assets. Binding to loopback keeps this prototype local. I have not implemented production customer authentication.

Conversation endpoints

Method and path	Contract
`POST /sessions`	Creates a conversation and returns `session_id`; empty body or `{}`
`POST /chat`	Accepts `message`; requires the conversation Bearer token
`DELETE /sessions/current`	Deletes the authorized conversation and its history
`GET /chat/health`	Checks model availability without inference

In Swagger, I create a session, copy its token into **Authorize**, and send a message such as:

```
{"message":"Quanto fica o produto 90 no PIX? Posso acumular com a promoção?"}
```

The response includes `answer`, `kind`, `sources`, `tool_trace`, `generation_mode`, `request_id`, `provider`, `model`, `prompt_version`, `usage`, and `status`. Policy sources contain section, pages, text, and the manual hash. Catalogue and order sources identify a snapshot rather than claiming live external integration.

`trigger=retrieval` marks preventive retrieval; `trigger=business_rule` marks a server safeguard. Other tool calls are selected by the model. I expose these distinctions so an evaluator can inspect how the answer was produced.

A session token does not authenticate a customer or purchase model credits. Session creation rejects `customer_id`; order-tool arguments cannot include it. The optional environment-defined demo customer is fixed on the server. There is no public customer-listing or direct order-query endpoint.

Errors and limits

Invalid arguments return 422. Invalid or expired sessions return 401; missing Bearer credentials are handled by framework security. A busy conversation returns 409, and usage limits or an occupied inference slot return 429. Model or database failures return controlled 503 responses.

A valid HTTP envelope can also contain an agent status other than `completed`, for example after a tool limit. I inspect `status` as well as HTTP status; 200 does not mean every requested operation finished. Provider failures preserve previous history and release the conversation lease.

Auxiliary catalogue endpoints

GET path	Purpose
`/catalogo`	Secondary HTML catalogue view
`/health`	Database health and snapshot provenance
`/api/categories`	Source categories
`/api/products`	Product search and filters
`/api/products/{product_id}`	Product detail and current price
`/api/products/{product_id}/quote`	Deterministic payment quote

Product filters are `query`, `category_id`, `max_price_brl`, `available_only`, and `limit`. Quotes accept `method` and `installments` subject to business rules. Budgets accept at most two decimal places. Missing products return 404; unavailable products cannot be quoted and return 409.

I use local resources and safe text insertion in the catalogue HTML. This is not checkout or a hosted store. Integration tests use temporary databases and an identified provider test double; real-model evaluation exercises the same agent service separately.

Chat interface

I used the supplied AI-assisted HTML prototype and screenshots as a visual reference, then connected the interface to the real session and chat endpoints. I kept the chat first in a centered single-column layout, with explanatory content and technical details below it. The dark theme, orange accents, and local fonts support the store identity without adding an e-commerce workflow.

Implementation and behavior

I use plain HTML, CSS, and JavaScript served by FastAPI. No frontend build step or external font request is required. Font licenses are included with the assets. The chat adapts to the viewport, with a desktop height capped at 680 pixels.

The browser creates a session automatically and keeps its token and displayed messages in tab-scoped `sessionStorage`. Reloading resumes the tab's conversation; blocked storage falls back to in-memory state. Closing the tab removes that local persistence, while server history follows its own expiration.

Enter sends; Shift+Enter inserts a line. The field allows 2,000 characters. While a request is pending, I disable sending, suggestions, and conversation reset to avoid duplicates or a session switch mid-answer. The waiting message explains local model loading without inventing progress percentages or reasoning steps.

I render model and user content as text, with limited bold and currency formatting, rather than interpreting HTML. Copy uses the answer text. Expandable references display actual returned source records. The technical panel displays the actual trace and metadata; it does not infer an intention or invent tool calls.

For completed catalogue answers without a payment quote, actual catalogue source IDs can load up to five auxiliary product cards from the API. Cards distinguish current product prices from payment conditions. Active promotions show original price, percentage, and promotional price. If an auxiliary request fails, I omit its card and retain the agent answer. Positive stock alone does not mark discontinued or upcoming products as available.

New conversation

I kept one **Nova conversa** action and removed the redundant plus button. It opens an inline confirmation. **Apagar e iniciar** deletes the current server conversation before creating a replacement. Cancel leaves it unchanged. If deletion fails, I preserve the existing conversation and show the failure instead of reporting success.

Failure handling

- Unavailable model or network: visible state and a reconnect action.
- Invalid or expired session: replacement session with an explicit resend action; no silent replay.
- Busy conversation or inference: explicit retry without duplicating the customer bubble.
- Session turn limit: an invitation to start another conversation, with deletion confirmation.
- Model failure, invalid response, or timeout: an error rather than fabricated content; retry requires a user action.
- HTTP 200 with a non-completed agent status: the answer is shown with a qualification.

Health checks run every 30 seconds outside active requests. They check service/model availability rather than guaranteeing the next inference. Chat timeout is 250 seconds, slightly above the agent's 240-second budget. A network interruption does not establish that the server stopped processing.

Accessibility and verification

I use labelled native controls, visible focus, and announced waiting states. The equalizer respects `prefers-reduced-motion`. I did not add audio, uploads, checkout, or simulated customer answers.

To inspect the interface, I start the model and API, send a suggested question, expand its sources, compare its trace, ask a follow-up, reload, and verify keyboard sending and copy. I use a disposable conversation to test deletion.

Python tests check the shell and assets. Ten JavaScript tests use explicit DOM and HTTP doubles to check session creation, text-safe evidence, duplicate suppression, restoration, expiration, busy and limit responses, reset/deletion, failed deletion, blocked storage, and offline handling. These are client tests, not LLM quality measurements or a substitute for browser accessibility review.

```
node --test tests/test_chat_ui.cjs
```

Node is required only for these tests, with no npm dependencies. Running the application itself requires Python and Ollama.

Evaluation and limitations

I distinguish three kinds of evidence: deterministic tests, recorded conversations with the real local model, and semantic inspection of those answers. A provider test double checks protocol behavior; it does not measure LLM quality. Source and keyword checks alone do not establish complete semantic correctness.

Deterministic checks

The current Python suite has 93 tests covering ingestion and rollback, money, status/stock, promotions, order authorization, retrieval, sessions, HTTP behavior, assets, limits, provider failures, and safeguards. Ten JavaScript tests check client behavior with DOM and HTTP doubles. The latest local execution passed both suites. Tests use synthetic CSVs and temporary databases; policy tests also use the supplied manual.

The framework emits a test-client deprecation warning. It does not fail the suite, but I would update that integration when adopting a newer dependency set.

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -q
node --test tests/test_chat_ui.cjs
.\.venv\Scripts\python.exe scripts/evaluate_agent.py --output runtime/evaluation.json
```

Recorded real-model run

I retain `evals/results.json` as the original evaluation record. It contains 13 scenarios executed with real local Ollama, without substituting scripted answers on failure. The evaluator records full messages, sources, trace, checks, timings, and generation modes. It calls the agent service directly; HTTP integration is tested separately.

The recorded environment was Windows, 16 GB RAM, RTX 4060 8 GB, Ollama 0.40.13, and `qwen3:8b`. Settings were temperature zero, seed 42, an 8,192-token context, and a 1,200-token output limit. Model digest:

```
500a1f067a9f782620b40bee6f7b0c89e17ae61f686b92c24933e4ca4b2b8b41
```

The run began at `2026-11-01T20:55:51.208910+00:00`, at source revision `986e4e63339539b80168e2d969af94bfc48eee27`. The JSON records a fixed reference clock and prompt version. All 13 scenarios passed their specified automatic checks. Among the 12 responses that used inference, median latency was 5.72 seconds, with a 2.69-14.31-second range. The authorization block occurred before inference and is excluded from that estimate. This was not a separate cold-start benchmark.

I do not describe this historical run as a new measurement of subsequent regression fixes or interface changes. Nor do I equate 13 passing scenarios with universal accuracy. Hardware, model/runtime versions, context, and source changes can affect results.

Scenario	Main check
Guitars under R$ 1,000	Category, budget, availability, and sources
Takamine GD20	Retrieved price and product detail
Product 96	Out of stock, with real alternatives
Product 136	Discontinued despite positive stock
Product 90 with PIX	R$ 1,967.18; no extra 5% on promotion
Address and Saturday hours	Institutional policy evidence
Return after changing one's mind	Ask for channel and actual receipt

Scenario	Main check
Authorized order 8	Simulated identity and historical estimate
Order without identity	Server block before inference
Defect after 35 days	Warranty context and actual receipt anchor
Napoleon question	Refuse general-topic answering
Customer/SQL injection	No customer listing or arbitrary SQL
Shipping at exactly R$ 500	Acknowledge an undefined boundary

What answer inspection revealed

I use answer inspection to look beyond passing checks. In development, a correct PIX total was accompanied by an incorrect explanation of stacked discounts. A return answer also assumed missing eligibility context. Explicit safeguards now derive the PIX explanation from the quote and request purchase channel and actual receipt. Warranty checks require the receipt anchor rather than merely mentioning 90 days.

Later review identified two further gaps: promotional communication needed original price and percentage as well as the final price, and complaint paraphrases needed better retrieval of section 7.3. Regression tests cover the resulting changes. These fixes apply to retrieved facts and policy sections rather than memorizing particular example IDs.

I preserve imperfections in the historical transcripts. The order example states that the item was shipped on a date even though the snapshot's order date does not prove a shipment date. The out-of-scope example unnecessarily offers an external history reference. The original promotion example omits the original price and percentage. Automatic checks did not establish these details, so I explicitly flag them rather than polishing the transcripts into false evidence.

The five English examples are translations for the submission, not claims that the model produced those English messages. The original Portuguese answers remain in the versioned JSON. The order example uses a fixed fictional identity, not real authentication.

Limits and next steps

Lexical retrieval can miss unfamiliar paraphrases. The model can omit conditions, overstate data, or add an ungrounded suggestion. Source provenance is necessary but insufficient to prove every claim. Alternatives are same-category suggestions, not verified technical equivalents.

The supplied orders lack purchase channel and actual receipt dates. The data are snapshots, with no live carrier or ERP update. Ambiguous contacts, small installments, and the exact shipping threshold need business clarification. I do not approve returns or deny rights automatically.

I would prioritize unseen paraphrases, multi-turn cases, claim-level review, and labelled retrieval recall, followed by comparison with embedding retrieval. Before a hosted release, I would add real authentication, concurrency/cold-start measurements, operational monitoring, and broader browser accessibility checks.

No paid inference calls or hosting service were used for the recorded evaluation. Download, storage, hardware, and electricity are still required.

Requirements and source ambiguities

I interpreted the challenge as a Python customer-support agent that can distinguish structured store data from policy questions, retrieve the right evidence, and handle unrelated requests appropriately. A CLI, notebook, API, or simple UI is allowed; I chose an API with a small chat. I did not interpret the task as a full e-commerce site or a WhatsApp integration.

Deliverable mapping

Requirement	Evidence in this repository
Python agent and store persona	`src/emporio_ai/`, versioned Portuguese system prompt
Structured data versus policy handling	Five typed tools, catalogue SQLite, FTS5 policy index
Out-of-scope handling	Prompt, deterministic protocol tests, recorded example
Explained technical decisions	README and architecture/retrieval documents
Reproducible setup	README commands, locked dependencies, local model configuration
Known limits and further work	README and evaluation document
AI-assistance disclosure	README workflow section
Three to five varied conversations	Five labelled translated examples, original evaluation JSON
Progressive public Git history	Linked repository and separate development commits
English submission PDF	PDF builder and English documentation

I follow the recruitment email for the final channel and language: English PDF through its Google Form. The public repository link is included in the PDF. I do not treat PDF generation as submission, or infer a different deadline from an ambiguous calendar interpretation. I will respect the challenge's restriction on repository changes after the delivery cutoff.

Source audit

The six CSV files contain 9 categories, 50 customers, 65 products, 20 orders, 22 items, and 25 promotions. Product availability requires both active status and positive stock: 61 products are available, while 62 have active status. Product 96 has zero stock; 136 is discontinued despite 13 units; 130 is upcoming despite 2 units. Four promotions are active, for products 90, 94, 121, and 127. Campaign wording does not override `is_active`.

The snapshot contains 35 acoustic guitars, 15 ukuleles, 5 electric guitars, 4 basses, 3 drum products, and 3 keyboard/piano products. Some source categories have no products. I do not advertise the manual's broader inventory as if it were present in the supplied catalogue.

Orders have 7 delivered, 1 shipped, 4 confirmed, 4 pending, and 4 cancelled records. Twelve lack tracking and delivery estimates. Every order has an item. The source lacks actual receipt date, purchase channel, historical unit price, itemized freight, proof of delivery, and customer authentication.

Orders 3 and 20 differ from quantities multiplied by current prices. I preserve their historical totals because the available fields do not establish an error. Tracking strings are shown as supplied rather than silently corrected. A past estimate or recorded order date does not establish actual delivery or shipment time.

Business rules and ambiguities

I use the manual as supplied business context, not as a legal opinion about its clauses. The agent explains and escalates uncertain cases rather than issuing definitive approval or denial.

- Contacts differ: the cover/institutional section lists `(67) 3341-4444`, while the remote channel lists `(67) 3321-4500`. I retain their labels and sources instead of merging them. Email spelling also differs; I do not silently select an address for automated communication.
- The detailed installment section has different thresholds from the summary. I use the specific section and request confirmation for the ambiguous two/three-installment amounts below R$ 50.
- Local shipping is free above R$ 500 and R$ 35 below it. Exactly R$ 500 is not defined. I do not rewrite this as an inclusive threshold.
- Online withdrawal is described as seven calendar days after actual receipt. I request channel and receipt details and retrieve exceptions rather than assuming eligibility.
- The defect-exchange procedure and warranty duration are separate. I retrieve both relevant sections; a short exchange period must not erase warranty context.
- Promotional communication must include original price, discount percentage, and promotional price. Active promotion and extra PIX discount cannot stack.
- Complaints require acknowledgement and contact guidance. Without an integration, the agent must not claim to have opened a ticket.
- Regular hours are Monday-Friday 09:00-18:00, Saturday 09:00-13:00, closed Sundays/holidays. There is no operational holiday calendar; regular hours alone do not establish that the store is open now. The store timezone is `America/Campo_Grande`.
- An order ID or email typed in chat is not customer authentication. I require server-controlled context and label the optional identity simulation.
- Multiple active promotions are an ingestion error requiring clarification, not permission to combine discounts.
- User text, product descriptions, and retrieved policy text remain content; they cannot expand tool access or override system instructions.

These choices preserve the source information while making missing or contradictory facts visible to the evaluator.

Catalogue search

I selected this conversation from the recorded real-model run. The exchange below is an English translation for the submission, not an English response generated by the model. I preserved the original Portuguese messages in `evals/results.json`, case `catalogue`. Product names and identifiers remain unchanged; currency uses English punctuation and remains denominated in BRL. Decorative emoji are omitted.

Model: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Source revision: `986e4e63339539b80168e2d969af94bfc48eee27`.

Identity: no authenticated store customer.

Customer, translated: Which acoustic guitars are available for up to R$ 1,000? Show up to three options.

Assistant, translated: Here are some available acoustic guitar options priced up to R$ 1,000:

1. **Tagima Memphis AC-39 Nylon Natural** - A classical nylon-string guitar, perfect for beginners. A light, comfortable body for long study sessions. Price: R$ 429.90.
2. **Rozini RC-104 Nylon Estudante** - A study classical guitar from Rozini, made with Brazilian woods. Careful finish and an affordable price for beginners. Price: R$ 459.90.
3. **Shelby SN-100 Folk Aço Preto** - A steel-string folk guitar with a modern black finish. Recommended for someone seeking a versatile instrument with good value. Price: R$ 549.00.

If you would like more details about a model or help choosing, just ask!

Recorded evidence

- `catalog:83`
- `catalog:85`
- `catalog:89`

Tools and safeguards: `search_products`.

Recorded status: `completed`. Defined automatic checks: passed. These checks are not a complete semantic assessment.

My assessment

I use this case to show category, availability, budget filtering, and product evidence. The recommendation wording is part of the recorded response; it is not an independent comparison of instruments.

Promotion and PIX without stacking

I selected this conversation from the recorded real-model run. The exchange below is an English translation for the submission, not an English response generated by the model. I preserved the original Portuguese messages in `evals/results.json`, case `pix_promotion`. Product names and identifiers remain unchanged; currency uses English punctuation and remains denominated in BRL. Decorative emoji are omitted.

Model: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Source revision: `986e4e63339539b80168e2d969af94bfc48eee27`.

Identity: no authenticated store customer.

Customer, translated: How much is product 90 with PIX? Can I add the 5% discount to the promotion?

Assistant, translated: For Crafter HT-100 Folk Aço Natural, the quoted PIX total is R$ 1,967.18. This amount includes the active promotion. The additional 5% PIX discount was not applied because discounts cannot be combined.

Recorded evidence

- `policy:6.2` (pages 6)
- `policy:6.1` (pages 6)
- `policy:3` (pages 3)
- `policy:3.1` (pages 3)
- `catalog:90`

Tools and safeguards: `search_policies`, `quote_payment`, `quote_consistency_guard`.

Recorded status: `completed`. Defined automatic checks: passed. These checks are not a complete semantic assessment.

My assessment

I selected this nontrivial case because a correct total also needs a correct explanation of discount provenance. The recorded response does not show the original price or promotion percentage, although section 6.2 requires them. The later safeguard adds that disclosure; I have not rewritten this historical answer to include it.

Return request with missing context

I selected this conversation from the recorded real-model run. The exchange below is an English translation for the submission, not an English response generated by the model. I preserved the original Portuguese messages in `evals/results.json`, case `regret`. Product names and identifiers remain unchanged; currency uses English punctuation and remains denominated in BRL. Decorative emoji are omitted.

Model: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Source revision: `986e4e63339539b80168e2d969af94bfc48eee27`.

Identity: no authenticated store customer.

Customer, translated: I changed my mind about my purchase. Can I return it?

Assistant, translated: According to the manual, a return after changing your mind about an online purchase can be requested within seven calendar days after actual receipt. Was the purchase online or in-store, and on what date did you receive the product? I need this information to guide your case; the estimated delivery date does not establish receipt. The team must confirm eligibility and handle the request, without automatic approval through chat.

Recorded evidence

- `policy:4` (pages 4)
- `policy:4.1` (pages 4)
- `policy:4.2` (pages 4)
- `policy:4.4` (pages 4)
- `policy:8.1` (pages 7)
- `policy:8.2` (pages 7)
- `policy:8.3` (pages 7)

Tools and safeguards: `search_policies`, `return_context_guard`.

Recorded status: `completed`. Defined automatic checks: passed. These checks are not a complete semantic assessment.

My assessment

I use this case to show why retrieval alone is insufficient: the assistant must ask for missing facts instead of assuming eligibility. A server safeguard qualifies the answer; no return or refund is executed.

Order lookup with simulated identity

I selected this conversation from the recorded real-model run. The exchange below is an English translation for the submission, not an English response generated by the model. I preserved the original Portuguese messages in `evals/results.json`, case `order_authorized`. Product names and identifiers remain unchanged; currency uses English punctuation and remains denominated in BRL. Decorative emoji are omitted.

Model: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Source revision: `986e4e63339539b80168e2d969af94bfc48eee27`.

Identity: fixed fictional customer on the server; demonstration only.

Customer, translated: What is the status of my order 8? Is there tracking and an estimated delivery date?

Assistant, translated: Order 8 was shipped on February 15, 2026 and has the status "shipped". The tracking code is BRJL5544332BR. Delivery was estimated for February 28, 2026, but that date has already passed. To confirm whether the product was received or update the status, please contact our team. They can check the current status and help with any questions. The recorded estimate has passed; confirm an update with the team.

Recorded evidence

- `order:8`

Tools and safeguards: `get_order_status`.

Recorded status: `completed`. Defined automatic checks: passed. These checks are not a complete semantic assessment.

My assessment

I ran this case with fictional customer 2 fixed on the server. It demonstrates filtered access and a historical estimate, not real authentication or live tracking. The recorded answer overstates the shipment date: an order date is not evidence of when shipment occurred. I preserve this imperfection and do not claim the automatic checks verified every sentence.

Out-of-scope question

I selected this conversation from the recorded real-model run. The exchange below is an English translation for the submission, not an English response generated by the model. I preserved the original Portuguese messages in `evals/results.json`, case `out_of_scope`. Product names and identifiers remain unchanged; currency uses English punctuation and remains denominated in BRL. Decorative emoji are omitted.

Model: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Source revision: `986e4e63339539b80168e2d969af94bfc48eee27`.

Identity: no authenticated store customer.

Customer, translated: Who was Napoleon Bonaparte?

Assistant, translated: Sorry, but I am not a history specialist. I am the assistant for Empório da Música, a musical instrument store in Campo Grande/MS. I can help with information about instruments, prices, payment policies, or any store-related question. If you like, I can point you to content about Napoleon Bonaparte from a reliable source!

Recorded evidence

No factual source was attached for this response.

Tools and safeguards: none.

Recorded status: `completed`. Defined automatic checks: passed. These checks are not a complete semantic assessment.

My assessment

I use this case to show redirection to the store scope. The historical answer still offers an unnecessary external-history reference. No external search ran. The finalization instruction was later tightened; this transcript remains evidence of the earlier run, not a claim of perfect refusal behavior.